2771N

Continuous Pursuit-Evasion with Aerodynamic Constraints

Computer Science Tripos – Part II

April 21, 2026

Declaration of Originality

I, the candidate for Part II of the Computer Science Tripos with Blind Grading Number **2771N**, hereby declare that this report and the work described in it are my own work, unaided except as may be specified below, and that the report does not contain material that has already been used to any substantial extent for a comparable purpose. In preparation of this report, I adhered to the Department of Computer Science and Technology AI Policy. I am content for my report to be made available to the students and staff of the University.

Date: April 21, 2026

Proforma

Candidate number:	2771N
Project Title:	**Continuous Pursuit-Evasion with Aerodynamic Constraints**
Examination:	Computer Science Tripos – Part II, 2026
Word Count:	WORD COUNT[1]
Code Line Count:	LINE COUNT[2]
Project Originator:	The candidate
Day-To-Day Supervisor:	Ramsey Faragher
Marking Supervisor:	Alastair Beresford

Original Aims of the Project

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Fusce ac turpis quis ligula lacinia aliquet. Mauris ipsum. Nulla metus metus, ullamcorper vel, tincidunt sed, euismod in, nibh.

Work Completed

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Fusce ac turpis quis ligula lacinia aliquet. Mauris ipsum. Nulla metus metus, ullamcorper vel, tincidunt sed, euismod in, nibh.

Special Difficulties

None.

[1] This word count was computed using `texcount`.
[2] This code line count was computed using `tokei` (excluding autogenerated test output).

Contents

1 Introduction **1**

1.1 Motivation 1

1.2 Project Aims 1

1.3 Success Criteria 2

2 Preparation **3**

2.1 Related Work 3

2.2 Starting Point 3

2.3 Theory and Design 4

2.4 Requirements Analysis 5

2.5 Development Methodology 5

3 Implementation **7**

3.1 Implementation Section 1 7

4 Evaluation **8**

4.1 Functional Evaluation 8

4.1.1 Validity Definitions 8

4.1.2 Observations 9

4.1.3 Parameter Tuning 9

5 Conclusions **10**

5.1 Conclusions Section 1 10

Bibliography **11**

A ⟨Sprint Reviews⟩ **12**

B ⟨Configuration Values⟩ **13**

C ⟨Validation Scenarios⟩ **14**

D Proposal **15**

Chapter 1

Introduction

1.1 Motivation

Pursuit-evasion problems model scenarios where agents compete to intercept or evade one another. They are a well-studied problem in aerospace systems research, with practical applications in autonomous air combat, unmanned aerial vehicle (UAV) swarm coordination, and airspace collision avoidance.

Approaching real-time solutions to these problems is challenging: the state space of even a simple scenario is continuous and high-dimensional. Exact solutions are computationally intractable[1], but Markov Decision Process-based formulations provide a performant framework for sequential decision-making through efficient approximation[2], and have been shown to scale to multi-agent teams in continuous 3D state spaces[3].

This project implements and extends the FastMDP algorithm of Bertram et al.[2] in a 3D kinematic simulation of fixed-wing aircraft. The primary focus is pursuit-evasion behaviour, investigated through novel reward functions and performance analysis. A key contribution is demonstrating that the same simulation and decision-making framework generalises to other problem spaces entirely, including collaborative flight and search-and-rescue, with changes only to the reward structure.

The project is implemented in Python 3.13.2, using `numpy` and `numba` for numerical computation.

1.2 Project Aims

The core aims of this project are to:

1. Implement a kinematically accurate 3D simulation of fixed-wing aircraft using a pseudo-6DoF model.
2. Implement the FastMDP algorithm from scratch to drive behaviours between competing or collaborating agents.
3. Design and investigate reward functions that produce valid pursuit-evasion behaviour.
4. Analyse the engineering performance of the simulation and decision-making framework.

After completing the core project, the following extensions were investigated:

5. Scale the simulation from 1v1 encounters to multi-agent teams of arbitrary size.
6. Demonstrate that the simulation and decision-making framework is applicable to alternative problem spaces beyond pursuit-evasion, requiring changes only to the reward structure.

1.3 Success Criteria

The following criteria were used to evaluate the success of the project:

1. **Behavioural validity**. The implemented FastMDP algorithm produces valid pursuit-evasion behaviour, defined according to a series of per-timestep constraints. This is the primary functional goal of the project and directly validates the correctness of the reward functions and decision-making framework.

2. **Kinematic validity**. Agent kinematics remain within physically realistic bounds throughout all simulation runs, including velocity, turn rate and altitude constraints. This validates the correctness of the kinematic model independently of the decision-making layer.

3. **Computational performance**. Agent actions are computed within a timeframe suitable for real-time simulation with appropriate optimisations investigated and evaluated.

4. **Extension validity**. Multi-agent teams and alternative problem spaces produce valid and consistent behaviour under the same framework.

The remainder of the dissertation covers the preparation, implementation and evaluation carried out to meet these criteria.

Chapter 2

Preparation

2.1 Related Work

Pursuit-evasion problems - where agents compete to capture or escape one another - were first formalised through a framework for reasoning about intelligent adversaries by Isaacs[4] and remain an active area of research across robotics, autonomous systems and aerospace[5].

Determining optimal strategies in real-time or close-to real time is computationally challenging as the problem grows in dimensionality - the state-action space becomes too large to represent with traditional exact methods[6, 7]. This 'curse of dimensionality'[8] motivates a variety of approximation approaches. McGrew et al.[1] demonstrated an approximate dynamic programming (ADP) approach to 1v1 air combat, involving a number of additional reward-shaping techniques. While effective, this approach is limited to 2D level flight at fixed velocity, does not approach the infinite state space of continuous flight[3], and does not scale to multi-agent teams. These limitations motivate a different approach for multi-agent pursuit-evasion in a continuous 3D environment.

Bertram et al.[2] proposed a novel algorithm to closely approximate the solution to a class of MDPs with sparse rewards which they term FastMDP. Instead of well-known value iteration or policy iteration methods[8, 9], FastMDP reframes the MDP as a graph and computes the value function from individual reward sources, combining positive and negative peaks into a single surface. The algorithm has no dependency on the size of the state space, making it suitable for continuous spaces and real-time computation. This was initially demonstrated in a 2D UAV path planning context, showing scaling with the number of obstacles.

Further works by Bertram and Wei[3] expanded FastMDP to a fully continuous 3D state space, incorporating a pseudo-6DoF kinematic model of fixed-wing aircraft. That work forms the basis for the simulation and decision-making framework developed in this project.

[REVIEW: NECESSARY?] Paragraph/justification about moving forwards with the FastMDP influence in my implementation, but what were my differentiations? Primarily reward structure, performance analysis alternative behaviour extensions

2.2 Starting Point

I had experience with building and applying Markov models in Python (in particular Markov Chains and Hidden Markov Models) from my Part IA Machine Learning and Real-World Data course, but I had not specifically implemented a Markov Decision Process or the FastMDP algorithm before.

I did not have experience with pursuit-evasion problems or explicit kinematic models, but I had written Java code to simulate physical three-dimensional systems before with a focus on linear algebra and vector mathematics rather than aerodynamics.

I was confident with the Python programming language and the tooling around it including virtual environments, the pip package manager and the pytest testing framework. I had real-world corporate experience in managing large python projects, and was also comfortable working with the Git version control system and GitHub actions for development workflows.

The final implementations are not based on existing codebases or machine learning libraries, but I identified similar projects from related work and examined their algorithms and pseudocode to influence my work.

2.3 Theory and Design

[TODO: THIS] Talk about the complexity of kinematic modelling as a "substantial body of theory" to understand and demonstrate additional challenge. Introduce MDPs first, then approximations of MDPs I ended up using, then finally FastMDP algorithm and my reward functions. Mathematical models and algorithmic descriptions. Lots of fun stuff to talk about, now I have a good sense of the algorithm etc. make sure it references how I understood the algorithm and required forms before jumping into implementation. MDPs FastMDP Rewards - negative vs positive, standard positive form, radii calculation Kinematics - TODO Talk about what I did instead of jumping straight into writing code - reference the papers and learning materials I used to understand MDPs, show my architecture diagram for the codebase (and then later discuss how this changed throughout implementation). What design decisions did I make early on? Pursuers vs evaders vs mutli-agents etc? Similar to above, codebase design decisions - and TOOL justificateionts!. Full OOP vs half-OOP with vectorised arrays. Very clear structure of the simulation vs optimisation with less abstraction opportunities. numba doesn't lend itself well to OOP (or most python-writing style) so that extension required some restructuring. Manager class was still used, but I extracted key optimisation methods like kinematic calculations for JIT compilation and utilised arrays instead of classes to represent agents for vectorised computation. It also meant that I wouldn't have to serialise/deserialise between Agent classes and raw data for forward projection. Other additional complexities could be numba library, talking about additional challenges involved in learning in-depth compilation strategies for python and how these can be translated to JIT C compilation and how this limits normal python stdlib. How did I know it would be a good fit? Need to motivate that I explored the components enough to know it would work Markov Decision Process Settlted on python because - experience working with it, numpy and numba for efficient computation, comfortable with libraries like matplotlib or pyvista for visualisations. Dynamically typed so sutitable. Anticipated challenges at the start: visualisations, performance/scalability, scope creep? For each of these describe how I anticipated and explored to make sure there were no hidden surprises. dfasd

asdfasd

asdfadsf

adsfsd

2.4 Requirements Analysis

Before beginning the project I performed a requirements analysis using the *MoSCoW* system. The *Must* requirements map directly onto the (non-extension) success criteria defined in the Introduction chapter.

Requirement	**Priority**
3D kinematic simulation of fixed-wing aircraft using a pseudo-6DoF model	`Must`
FastMDP-based decision making framework for pursuit-evasion behaviour	`Must`
Agent actions are computed within a timeframe suitable for real-time simulation	`Must`
Valid kinematic and behavioural outputs verified against defined criteria	`Must`
Output system producing video or visualisation results of simulation runs	`Should`
Reward functions are configurable without changes to core implementation	`Should`
Codebase is modular and extensible to alternative problem spaces	`Should`
Scale to multi-agent teams beyond 1v1	`Could`
JIT compilation via `numba` for performance gains	`Could`
Demonstrate framework applicability to alternative problem spaces	`Could`
Additionally accurate kinematic modelling beyond pseudo-6DoF	`Won't`
Real-world hardware deployment	`Won't`

Table 2.1: MoSCoW requirements analysis

2.5 Development Methodology

The project was developed on a personal MacBook Pro (2021, Apple M1 Pro, 16GB RAM) running Python `v3.13.2`. Key libraries were verified for compatibility before implementation with particular attention to `numba`, which supports ARM-based Apple Silicon[10] but required testing to confirm JIT compilation behaved correctly.

Git was used for version control throughout development, with source code, assets and dissertation drafts regularly committed to a remote GitHub repository.

An agile approach to software development was adopted throughout the project. The milestones from the project proposal were used as starting points for implementation sprints, with a short written review after each 2-3 week sprint. These reviews are included in Appendix A. This approach allowed flexibility throughout the academic year as other in-term commitments and vacations affected the development schedule, which is also reflected in the commit frequency of the repository.

Unit tests were generated for individual modules using the `pytest` framework and produced alongside each implementation. A GitHub Actions workflow was configured to run the full test suite on every remote commit, with results and coverage reports stored as build artefacts. This

provided a record of test outcomes throughout development to be referenced in the Evaluation chapter. However maintaining high test coverage proved to be challenging during heavy refactoring of the central implementations and limited the effectiveness of the suite. This is discussed further in the Implementation chapter.

Generative AI tooling was used in two clearly delineated capacities. Firstly, it was used to generate the unit tests from the existing implementations, coming up with realistic and edge-case values for the simulation and kinematic components. Secondly, it was used to produce boilerplate code for the output and visualisation modules with the Matplotlib library. All simulation, MDP, and kinematic code was written independently. No AI-generated code was included in the main algorithmic implementations, but separate chat windows were used throughout development to aid with documentation searching and reinforcing theoretical knowledge.

Chapter 3

Implementation

Implementation

3.1 Implementation Section 1

d

Chapter 4

Evaluation

The success criteria outlined in the Introduction chapter were revisited to inform evaluation.

1. **Behavioural validity**. The implemented FastMDP algorithm produces valid pursuit-evasion behaviour, defined according to a series of per-timestep constraints.
2. **Kinematic validity**. Agent kinematics remain within physically realistic bounds throughout all simulation runs, including velocity, turn rate and altitude constraints.
3. **Computational performance**. Agent actions are computed within a timeframe suitable for real-time simulation with appropriate optimisations investigated and evaluated.
4. **Extension validity**. Multi-agent teams and alternative problem spaces produce valid and consistent behaviour under the same framework.

4.1 Functional Evaluation

Throughout implementation I had been iteratively adjusting the FastMDP algorithm and reward functions to drive desirable behaviour. As described in the Implementation chapter this process was driven partly by qualitative analysis of the flight paths and partly by manual simulated annealing of my reward function parameters. After completing the project core, I wanted to more strictly examine the pursuit and evasion behaviour of the agents to inform the success of the project.

I designed a set of 1v1 validation scenarios, each defined by a specific combination of initial values and agent parameter ratios, ranging in complexity. Each scenario was run for up to 1000 timesteps or until a capture, and at each timestep the agent values were recorded and checked against predefined validation criteria. A scenario is considered valid if all tracked values satisfy these criteria throughout the run. The exact values of the scenarios and their validation results can be found in Appendix C.

4.1.1 Validity Definitions

I defined two types of validity and the criteria for them between timesteps.

Kinematic Validity

For each agent for each timestep, three parameters were checked against the bounds established in the kinematic model:

- Velocity $v \in [34.3, 103.0]$ m/s (0.1 - 0.3 Mach)
- Azimuth rate $\dot{\psi} \in [-1.3, 1.3]$ rad/s
- Altitude $z > 0$ m

Behavioural Validity

Let $d(t) = ||\mathbf{p}_1(t) - \mathbf{p}_2(t)||$ be the Euclidean distance between a pair of agents with position functions $\mathbf{p}_1$ and $\mathbf{p}_2$.
Further let $d_0 = d(0)$ the initial separation and $d_{\min} = \min_{t \in [0,1000]} d(t)$ the minimum separation.

A scenario is considered behaviourally valid if:

1. $\exists\, t.d(t) < d(t-1)$ — i.e. $d(t)$ does not monotonically increase throughout the run.
2. $d_{\min} < d_0$ OR capture occurs during the run.

Together these ensure that the reward functions produce meaningful interaction – a non-increasing $d(t)$ at some point implies at least one agent is closing on the other's position, and $d_{\min} < d_0$ confirms this response results in net closure over the course of the run.

If all agents are kinematically valid, and all pairs of agents are behaviourally valid, then the scenario is valid.

4.1.2 Observations

I plotted the kinematic and ehavioural validites! examples here?

Note that in scenario 1, agents initialised head-on at large separation may satisfy these criteria through near-straight-line flight, which is also optimal behaviour in that configuration so it is not particulary useful to examine the pursuit-evasion behaviour stuff. This scenario therefore primarily validates kinematic correctness rather than complex decision-making.

in scenario 2, the agents face directly away from each other. You might think they maximise their turn rates, but instead use a combination of upward lift and slight turns to fall back towards each other?

Scenario 7 - kinematically failed! went below hard deck - my penalty clearly didn't work.

4.1.3 Parameter Tuning

Chapter 5

Conclusions

Conclusions...

5.1 Conclusions Section 1

Conclusions Section 1...

Bibliography

[1] James S. McGrew et al. "Air-Combat Strategy Using Approximate Dynamic Programming". In: *Journal of Guidance, Control, and Dynamics* 33.5 (2010), pp. 1641–1654.

[2] Josh Bertram et al. "Online Flight Planner with Dynamic Obstacles for Urban Air Mobility". In: *AIAA Aviation 2019 Forum*. 2019. DOI: 10.2514/6.2019-3625.

[3] Jason Bertram and Jianghai Wei. "Scalable Online Planning for Multi-Agent MDPs". In: *arXiv preprint* (2019). arXiv: 1909.04171.

[4] Rufus Isaacs. *Differential Games*. Wiley, 1965.

[5] Isaac E. Weintraub, Meir Pachter, and Eloy Garcia. "An Introduction to Pursuit-Evasion Differential Games". In: *arXiv preprint* (2020). arXiv: 2003.05013.

[6] Dimitri P. Bertsekas and John N. Tsitsiklis. *Neuro-Dynamic Programming*. Athena Scientific, 1996.

[7] Warren B. Powell. *Approximate Dynamic Programming: Solving the Curses of Dimensionality*. Wiley, 2007.

[8] Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.

[9] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 1998.

[10] Numba Developers. *Installing Numba*. 2024. URL: https://numba.readthedocs.io/en/stable/user/installing.html (visited on 04/21/2026).

Appendix A

⟨Sprint Reviews⟩

Sprint reviews!

Appendix B

⟨Configuration Values⟩

Appendix B...

Appendix C

⟨Validation Scenarios⟩

Appendix C...

Appendix D

Proposal

Proposal...